

[Home](#) [News and blogs hub](#)

Lost and Frustrated but Persistent, part 2: personal narratives about usability challenges with Open Source Scientific Software



Bookmark this page

Lost and Frustrated but Persistent, part 2: personal narratives about usability challenges with Open Source Scientific Software

Author(s)



M. Doherty SSI fellow

A. Eggert

Y. Eid

K. Lehmann

C. Meesters

L. Schüler

D. Wicaksono

Posted on 4 July 2023

Estimated read time: 7 min

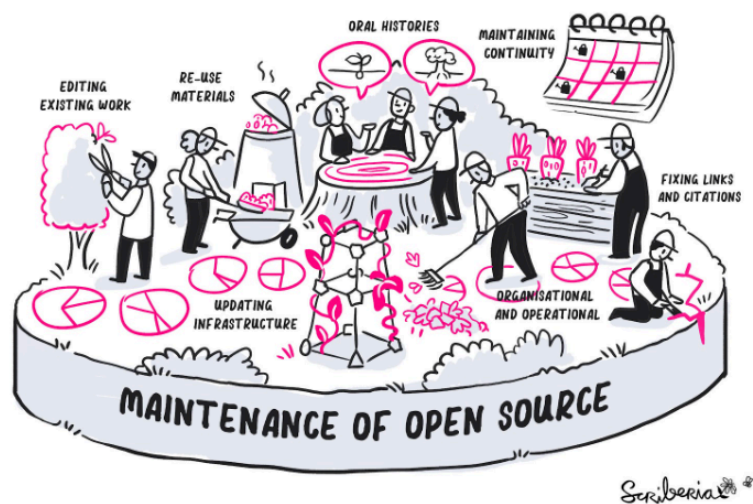
Sections in this article

The persistent biostatistician

The Altruistic Author



Posted by d.barclay on 4 July 2023 - 8:30am



This illustration is created by Scriberia with The Turing Way community. Used under a CC-BY 4.0 licence. DOI: 10.5281/zenodo.3332807

By SSI Fellow Meag Doherty, Anja Eggert, Yomna Eid, Kjong-Van Lehmann, Christian Meesters, Lennart Schöler, Damar Wicaksono.

This blog is the continuation of **part 1**.

The persistent biostatistician

Being the statistical consultant in a medium-sized research institute, I support the scientists in their data processing and statistical analysis. As I'm an advocate of open and reproducible science, the software equipment I need for my work is beyond statistical software. Using containers for reproducible workflows (like Docker or Singularity) has become a widespread approach. As a scientist, I see the many benefits of utilizing software containers. But I'm also aware that security risks come along when using it on an institute's server. As I see it, **it should be the task of IT to set up rules and to find a good balance between security and usability**. Unfortunately, the answer often is: "NO, we will not install Docker on the server because of security issues." Arguing with the IT engineers often feels unfair - I mean I'm a biologist and

statistician by training, I'm not a computer scientist or engineer - and I leave such conversations as a loser... I wish I could have argument papers ("How To convince the IT to set up Docker well-thought-outthought arguments.

The Altruistic Author

I work in an interdisciplinary academic setting where it is already a challenge to find a common vocabulary. On top of that, many people in my fields don't have a very strong background in computer science. As the author of a scientific software package, I quickly realized that **only documenting every single feature is not enough when facing such obstacles**. But step-by-step examples of how to use features of a piece of software can give a good starting point for other people's research endeavours. By seeing the results of small examples, users get a much better understanding of how to use the software. Besides the actual learning, examples can also be a neat way of explaining certain pitfalls or sensible value ranges.

There are **not many incentives for investing time and resources into such usability improvements**. But when users have questions or problems, it actually saves a lot of time when you're able to simply refer to an already existing example which shows how to solve the problem. Providing such a reference is surprisingly often enough help users need. Apart from that, getting feedback from new users who are happy because they could set up their research workflows quickly and without much frustration is great.

The not-so-unhappy Research Software Developer/Product Owner

I am working in Research Data Management of a medium-sized, non academic environmental research facility as a developer in several smaller software packages and the product owner of a larger-scale data infrastructure project. I am in the lucky situation to be a full-time software research developer, without the need to intermingle two very distinct career paths, namely being a scientist **and** a software developer.

Usability is an issue for us, with two distinct sides to it. On the one hand, it is an important measure, as user experience is one of the main driving factors

for the adoption of our products and services and adoption is one of the most important assessment criteria for our work. On the other hand, **we are lacking knowledge, experience and tools on how to assess and improve the usability of our artefacts.**

Currently, we are addressing the issue mainly by starting out with our own, but heavily developer and software design oriented ideas on the topic, bring a project into the state of a minimal viable product before we address the broader public with user/stakeholder workshops and introductory courses. While this approach is working so far, it is still rear facing and **addresses usability after the fact instead of putting it into the centre of the software development process.**

I guess, **what we really need is more internal and/or external formal knowledge on the topic.** Training for developers and probably also external consultants, working with the development teams during the critical phases of a project, would likely help a lot. However, I also think **this would mean another cultural shift in the science-software relationship before the necessary first one is completed.** We are still in a transition phase, where the idea, that the development of scientific software is a dedicated profession in its own right and not something scientists can or should do during their spare time, is not entirely settled yet.

So **we need to see a further professionalization of the software development process in science first** before we are able to finally integrate more of the ideas of the commercial software development processes into our own culture.

An anxious research software engineer outside the comfort zone

I'm currently working on the development and maintenance of a Python package for high-dimensional interpolation. There are recent results in this topic from applied mathematics that we think are beneficial to bring to the wider scientific audience. Although I'm working closely with mathematicians in the project, I'm not a trained mathematician myself. While this can be frustrating at times (due to my lack of some theoretical foundations), this difference in the background can be an asset as well, especially when it comes to designing public interfaces for the non-mathematicians audience.

There are papers, there are software implementations, and there are usable (to the target audience) software implementations. Creating bridges between them is non-trivial. Translating results from papers to (any) software implementation is probably the most straightforward for us. We write the software for ourselves and we use it ourselves to produce even more results. Even if the software is badly designed, one can get used to that and still be (somewhat) productive with it.

Creating a usable public software implementation, on the other hand, is tricky. It requires additional knowledge about the potential users, their needs, and their backgrounds. **All the public interfaces and the documentation must be well considered, not to mention taking into account users' feedback from different channels. We want people to use the software, after all.**

I think the effort that we put into usability depends on the goal of the project itself. If it's for internal consumption, and it's just about new features and producing the next results, then I think any ways of writing software are good enough as long as the product serves its (internal) purposes. But if one of the goals is also to disseminate the development to a wider audience, then it entails certain responsibility of adhering to the **community standards and best practices of usable open research software**. It's certainly more than, say, simply putting the source code on an online public repository. Adhering to those standards—from properly naming things to writing public documentation—takes time, and most probably would hinder the development of brand-new features.

It would be great if the goal of a (presumably open) scientific software project is clarified from the beginning: is it actually just for internal consumption within a small group or indeed for public consumption? If it's for the latter, then **usability should be part of the requirement**. The project lead should understand this, and together with the team, must juggle between different priorities: adding new features, maintaining the old ones, and developing more usable software.

*... Do you see yourself in these narratives? Do you have ideas on how to make things better? Join **the discussion and share your share experience**.*

Share on blog/article:





[Subscribe](#)

[Data management](#)

[Privacy policy](#)

[Web accessibility](#)

[Environmental Sustainability Policy](#)

[Contact](#)

[Site map](#)



©2010 - 2026 The University of
Edinburgh on behalf of the Software
Sustainability Institute.

Except where otherwise noted,
content on this site is licensed under
a Creative Commons Attribution
Non-Commercial 2.5 License.

